THE STATISTICS & SIMULATION TRAINER

or: Randomly Deviating in Value Per Page!

A field guide to making up numbers, on purpose

What's inside this thrilling page-turner:

0. How to Read This Thing ..... the ground rules
1. Random Numbers & the Seed ..... pinning the dice
2. The Repeating-Random Lab ..... the seed is the cause
3. The Average ..... add up, divide, done
4. The Gaussian ..... the bell's two dials
5. random vs. matplotlib ..... who makes, who draws
6. Variance & Standard Deviation .. measuring the spread
7. Simulating the Average ..... the Central Limit Theorem
8. Standard Error of the Mean ..... trusting the average
9. Confidence Intervals ..... the honest range
10. The Cheat Sheet ..... everything on one page

WARNING

Contains data generated entirely from nothing. No real pizzas were weighed. If your variance comes out suspiciously small, you divided by n. Divide by n minus one. The autograder is watching.

Master of Data Science - CMPINF-2100 - Module 4 Companion Notes

One notch above plain-beginner. No prior standard-deviation trauma assumed.

0. How to Read This Thing

Each section follows the same rhythm: the idea in plain words, the code or formula, a worked example you can trace by hand, and the trap that will bite you. **Read the traps twice.** The traps are where the points live.

Every topic in Module 4 serves one escalating question: **when you compute an average from a sample, how much can you trust it?** Random numbers hand you the sample. The average summarizes it. Variance and standard deviation measure how spread the data are. The standard error measures how spread the *average* is. A confidence interval turns that into an honest range. And simulation is the trick that lets you watch all of it happen using data you invent from nothing.

Three markers you'll see in every section

The Idea — the plain-English version, no jargon.

The Trace — a worked example with the actual numbers, one step at a time.

The Trap — the specific way this breaks. Usually a rewound seed, a wrong denominator, or a misread parameter.

One promise: every code sample here runs. Type it into a Jupyter cell exactly as written and it works. Comments marked with # are notes to you, not code that does anything. One house rule for this module: it is plain Python lists throughout, so the import is `import random`*, not* `import random as rd`*, and there is no NumPy yet.*

1. Random Numbers & the Seed

The Idea

`random.random()` hands you one number between 0 and 1, drawn from a **uniform** distribution: every value equally likely, no favorites. But it is not truly random. Under the hood it walks a long, fixed, pre-computed sequence of numbers that only *look* random. The **seed** is simply where in that sequence you start.

The Syntax

```
import random

random.random()      # one uniform draw, e.g. 0.764...
random.seed(2100)    # set the starting position (a bookmark)
random.random()      # always 0.764... from here
```

Think of the seed as a vending-machine code: punch in 2100 and you always get item A first, item B second, item C third. Same lineup, same order, every run. It pins the *whole sequence*, not one repeating value. That reproducibility is the entire point: it makes your screen match the lecture and makes a simulation gradeable.

The Trace

Set the seed once, then call twice. Each call walks one step forward:

step	call	value you get
seed	random.seed(2100)	(sets position, returns nothing)
1	random.random()	0.764021138054951
2	random.random()	0.4692037101382426

The two values differ from each other because each call advances. Re-run the program from `seed(2100)` *and you get 0.764… then 0.469… again, in that order, forever.*

The Trap: reseeding rewinds the sequence

Seed **once**, then call as many times as you want. Reseeding before every call sends you back to position one, so you get the same value over and over:

```
random.seed(2100); random.random()   # 0.764
random.seed(2100); random.random()   # 0.764 again -- rewound!
```

Pattern to keep: seed once, *then* loop. The one place reseeding-per-call is correct is inside a function that says 'this seed always yields this result' — for example `def roll(s): random.seed(s); return random.randint(1,6)`, where `roll(2100)` is always 3.

2. The Repeating-Random Lab

The Idea

This lab ships with no lecture, which is why it feels like nothing happens. Its whole job is to prove one thing by contrast: **unseeded randomness changes every run; seeded randomness stays the same.** The title means "*repeatable* random generation," not "generate numbers repeatedly in a loop."

The four beats

1. Random looks random — five varied values, the baseline. **2.** And it changes every run — the cell is labelled 'before Setting Seed' on purpose; run it twice and the numbers move. **3.** Set a seed, now it's pinned — nearly identical to beat 2 except for the seed line, so the seed is provably the only cause. **4.** Scale it to a dataset — seed once, build a list, rerun, get the same ten numbers.

The Trap: running it once hides the whole lesson

Beats 2 and 3 only differ on the *second* run. On a single top-to-bottom pass they look identical. The contrast (beat 2 moves, beat 3 doesn't) appears only when you re-execute. Two draws inside one run *should* differ from each other; what's reproducible is the sequence **across runs**, comparing same-position to same-position.

Side Quest: the underscore (_)

You'll see `for _ in range(5):` everywhere. In Python the _ is just a throwaway variable name — a convention saying "I need a loop but never use the counter." If you *do* use the index, give it a real name like `i`.

If you were warned to avoid it, that warning is about *other* languages, where the symbol has teeth:

Where	What _ means there
Python	convention only -- a throwaway name (use it freely)
Go	the blank identifier -- you MUST discard unused values with it
Rust	a true wildcard in pattern matching (a different mechanism)
Python REPL	holds the last result, so reusing it can clobber that

Same symbol, same intent ("ignore this"), different enforcement. In a notebook cell the underscore is fine and idiomatic.

3. The Average

The Idea

The average is the foundation the rest of the module stacks on: the sum of the values divided by how many there are. Nothing more.

The Syntax

```
def my_avg(x):
    """Returns the average of a list x"""
    return sum(x) / len(x)
```

The Trace

x	sum(x)	len(x)	sum / len
[2, 4, 9]	15	3	5.0

Why hand-roll it when `statistics.mean`, `numpy.mean`, *and pandas* `.mean()` *exist? Because variance calls the average, SD calls variance, and SEM calls SD. Writing the chain by hand once makes you see that variance is literally 'the average of squared distances from the mean.' In real work you'd call the library. (Aside on naming: the lab calls it* `my_avg`*;* `find_avg` *reads slightly off, since an average is computed, not found.)*

4. The Gaussian

The Idea

Gaussian = normal distribution = bell curve: three names, one thing. Unlike the flat uniform from `random.random()`, a Gaussian has a center where draws cluster and tails that taper off. You generate from it with `random.gauss(mu, sigma)`, and the two arguments are **two independent dials**:

- **mu = position (the mean).** Where the peak sits. Change it and the whole bell *slides* left or right; the shape never changes.
- **sigma = width (the standard deviation).** How fat or skinny the bell is. This is the *only* dial that touches shape.

The Syntax

```
random.gauss(mu=5, sigma=1)                      # one draw near 5
samples = [random.gauss(0, 1) for _ in range(1000)]   # a whole bell
```

The quiet payoff: `random.gauss(mu, sigma)` is literally `random.gauss(mean, standard_deviation)`. The two dials *are* the two statistics you compute by hand later. And a single draw is just one number — the bell only exists in the **collection**, which is why the lab draws a thousand and histograms them.

> **The Trap: four myths about gauss**
>
> - **"The number size sets the shape."** No — only `sigma` does. `gauss(1, 2)` is wide, not narrow.
> - **"`gauss(0,0)` is uniform."** The opposite. `sigma=0` is zero spread: every draw is exactly 0.0, a single spike. Uniform is a different distribution, reached with `random.random()`.
> - **"Extreme values make an upside-down bell."** Impossible: a histogram counts occurrences, and counts can't go negative, so the curve always points up.
> - **"mu vs sigma being equal matters."** Their relationship to each other is irrelevant; only the sigma value sets the width.

5. random vs. matplotlib

The Idea

A common mix-up is blending two different libraries. Keep them apart with one image: **random is the kitchen** (it makes the data); **matplotlib is the plating** (it arranges the data for viewing). The bell curve is not a matplotlib feature and `gauss` is not a matplotlib call — the bell is a shape that *emerges* when you histogram a pile of gauss values.

Call	Library	What it does
random.gauss(mu, sigma)	random	makes a number from a bell (NOT matplotlib)
random.random()	random	makes a uniform number on [0,1]
plt.hist(data)	matplotlib	draws data as a histogram (bars)
plt.plot(x, y)	matplotlib	draws data as a line

The Syntax

```
samples = [random.gauss(0, 1) for _ in range(1000)]   # random: make it
import matplotlib.pyplot as plt
plt.hist(samples); plt.show()                         # matplotlib: draw it
```

The command reference

Basic — the calls in your labs:

plt.hist(data)	histogram (bars); this is what makes a bell visible
plt.plot(x, y)	line plot; connects points
plt.title / xlabel / ylabel	title and axis labels
plt.legend()	show the key (needs label= on your plots)
plt.grid(True)	gridlines behind the plot
plt.show()	render the figure; ends the current plot

Advanced — the near-neighbors:

plt.scatter(x, y)	dots only, no connecting line
plt.bar(x, height)	bar chart for categories (not binned like hist)
plt.figure(figsize=(w,h))	set canvas size before plotting
plt.xlim / plt.ylim	set axis ranges by hand
plt.savefig('name.png')	write the figure to a file
plt.axvline / plt.axhline	a reference line (e.g. mark the mean)

Expert — reach for these when needed:

plt.subplots(r, c)	grid of plots in one figure
plt.tight_layout()	auto-space subplots so labels don't overlap
plt.annotate / plt.text	text (with optional arrow) at a coordinate
plt.axvspan / plt.axhspan	shaded band (e.g. mark a confidence interval)
plt.style.use('ggplot')	apply a preset look
plt.close()	close a figure (frees memory in big loops)

A frame for any new call: almost every `plt.something()` *either* ***draws the data*** *(hist, plot, scatter, bar),* ***labels/decorates*** *(title, xlabel, legend, grid), or* ***controls output*** *(show, savefig, figure). Sort it into one of those three and it clicks.*

6. Variance & Standard Deviation

The Idea

Variance and standard deviation measure how **spread out** the data are around the mean. Variance is the average of the squared distances from the mean; standard deviation is its square root, which brings you back to the original units. SD is the one people actually report — read it as a *typical* distance from the mean.

The Syntax

```
def my_variance(x):
    xbar = my_avg(x)
    squared = [(xn - xbar)**2 for xn in x]
    return sum(squared) / (len(x) - 1)    # NOTE the parentheses

def my_sd(x):
    return my_variance(x) ** 0.5          # ** 0.5 IS the square root
```

The Trace

Variance of `[48, 49, 50, 51, 52]`, whose mean is 50:

value	value - mean	squared
48	-2	4
49	-1	1
50	0	0
51	1	1
52	2	4
	sum of squares	10

*Then variance = 10 / (5 – 1) = **2.5**, and SD = √2.5 ≈ **1.58**. Note the denominator is 4, not 5 — that's the next idea.*

> **The Trap: the missing parentheses**
>
> These two lines are **not** the same:
>
> ```
> return sum(sq) / (len(x) - 1)    # RIGHT: divide by (n-1)  -> 2.5
> return sum(sq) / len(x) - 1      # WRONG: (sum / n) - 1     -> 1.0
> ```
>
> Python divides before it subtracts, so the second line computes `(sum / n) - 1` — a wrong number that still looks like a plausible variance. The –1 must live *inside* the denominator.

The Idea: why n minus one

Your data is a **sample** drawn from a larger population, and dividing by n systematically under-estimates the true spread. The fix (Bessel's correction) is 'degrees of freedom': with the mean of `[48,49,50,51,52]` fixed at 50, the five values must sum to 250. Fill in the first four freely and the fifth is **forced** to be 52. So five numbers carry only four free pieces of information — and that spent degree is the –1.

> **ddof, read literally (not as 'delta')**
>
> NumPy spells this `ddof` ("delta degrees of freedom"), but ignore the word delta — it only confuses. Read it as exactly one thing: **how much to subtract from n in the denominator** (denominator = n – ddof).
>
> ```
> # for n = 5:
> ddof = 0  ->  5 - 0 = 5   POPULATION variance  (numpy's default)
> ddof = 1  ->  5 - 1 = 4   SAMPLE variance      (the course; Bessel)
> ```
>
> Bigger ddof, smaller denominator, larger variance. In practice you only use two: **ddof=1** for samples (the course, and almost all real work) and **ddof=0** for a full population.

The library way: variance & SD

In real work you call a library, not `my_variance` — but the libraries **disagree on their defaults**:

Call	Denominator	Matches course?
statistics.variance / .stdev	n - 1 (sample)	yes, out of the box
numpy.var / numpy.std	n (population)	NO -- pass ddof=1
numpy.var(x, ddof=1)	n - 1 (sample)	yes, once corrected
pandas .var() / .std()	n - 1 (sample)	yes (helpful default)

Verified on that same list (true sample variance 2.5): `statistics.variance -> 2.5`, but `numpy.var -> 2.0` until you add `ddof=1`. When in doubt, use n–1.

7. Simulating the Average

The Idea

This video has no lab and a misleading title. The one thing to internalize: it is **all synthetic data — even the first value.** There is no real dataset, no file, no measurement. `random.random()` invents every number from nothing. That is what makes it **simulation**, not interpolation or extrapolation:

- **Interpolation** fills a gap *between* values you already have. (Needs real data.)
- **Extrapolation** projects *past* values you already have. (Needs real data.)
- **Simulation** *generates* the data from a random process, then watches the pattern. (Needs no data.)

The test for any situation: “if I delete the computer, does the data still exist somewhere?” For measured pizza weights, yes. For `random.random()` draws, no — they exist only because the code ran.

The Syntax

```
random.seed(100)
averages = []
for _ in range(1000):                              # repeat 1000 times
    sample = [random.random() for _ in range(5)]   # invent 5 'weights'
    averages.append(my_avg(sample))                # store THIS mean

my_avg(averages)   # 0.5035  -- lands on the true 0.5
```

The Trace

Histogram the 1,000 sample-means and a bell appears, peaked at ~0.5, even though the source was flat uniform:

```
 0.33 | ##################
 0.43 | ########################################
 0.48 | ############################################   <- peak, ~0.5
 0.53 | ########################################
 0.62 | ##############################
 0.73 | ############
```

That is the **Central Limit Theorem**: the distribution of sample means is approximately normal, whatever the shape of the underlying data. Intuition: for five draws to average near 0.0 they must *all* be tiny at once (rare), but there are many ways to land near 0.5. This bell is the hinge the rest of the module turns on — because sample means are Gaussian, you can put an error bar and a confidence interval on a mean.

8. Standard Error of the Mean

The Idea

SEM is the standard deviation of the **average itself** — how much your sample mean would bounce around if you took the sample over and over. Hold the distinction:

- **SD** measures the spread of the individual data points. “How scattered are my values?”
- **SEM** measures the spread of the mean. “How much can I trust my average?”

The Syntax

```
def my_sem(x):
    return my_sd(x) / len(x)**0.5    # SD divided by the square root of n
```

The Trace

Uniform data, seed 2100, each row reseeded so they’re comparable:

n	SD (spread of data)	SEM (spread of the mean)
5	0.2974	0.1330
20	0.2900	0.0649
80	0.2834	0.0317
320	0.2777	0.0155

SD barely moves (~0.29): more data doesn’t make individual values less scattered. SEM collapses, halving each time n quadruples.

- **SEM is always smaller than SD** (for any real sample). The average of several values is steadier than the values themselves.
- **Diminishing returns.** The term is 1/√n, not 1/n, so you must *quadruple* n to *halve* the SEM. That square root is why ‘just collect more data’ gets expensive fast.

> **The Trap: watch the two square roots**
>
> In `my_sd`, `** 0.5` takes the root of the *variance* to make SD. In `my_sem`, `** 0.5` is on `len(x)` — the sample size — and SD is divided by it. SD itself is never raised to a power in the SEM step.

9. Confidence Intervals

The Idea

A confidence interval is the **range** you report for where the true mean probably sits, instead of a single number: the ‘margin of error’ from polls. The formula leans entirely on SEM:

```
#  95% CI  =  mean  +/-  2 * SEM
sem = my_sd(sample) / len(sample)**0.5
ci  = (my_avg(sample) - 2*sem, my_avg(sample) + 2*sem)
```

SEM is the width of the interval. Big SEM, wide CI, ‘could be anywhere’; small SEM, narrow CI, ‘right around here.’ The **2** comes from the bell: about 95% of a normal distribution sits within ~2 standard deviations of the center, and the SEM is the standard deviation of the bell of means. (The exact multiplier is 1.96; 2 is the quick version the course uses.)

The Trace

Watch the interval tighten as the mean gets more trustworthy (true mean = 0.5):

n	sample mean	SEM	95% CI	confidence

5	0.394	0.1330	(0.128, 0.660)	low -- far too wide
1000	0.512	0.0091	(0.494, 0.530)	high -- tight on 0.5

At n=5 the wide interval still captures 0.5 — it's hedging honestly. At n=1000 it brackets 0.5 tightly.

The Trap: what '95% confident' actually means

It's tempting to say "95% chance the true mean is in *this* interval." Strictly, that's the one thing a CI does NOT say. The true mean is fixed; the interval is the random thing. What it really means is about the **procedure**: repeat the sampling many times, build a CI each time, and about 95% of *those* intervals contain the true mean.

Exam note: the course states it the loose way. Answer the course's way on quizzes; carry the precise version into real work.

The library way: SEM & CI

These live mostly in `scipy.stats` (the `statistics` and `numpy` modules have no direct SEM or CI function):

```
from scipy import stats
stats.sem(arr)                                  # SEM, matches my_sem
stats.norm.interval(0.95, loc=m, scale=sem)     # ~1.96 / '2-sigma' CI
stats.t.interval(0.95, n-1, loc=m, scale=sem)   # t-based CI (small n)
```

Without scipy, build SEM as `numpy.std(arr, ddof=1) / len(arr)**0.5` — note `ddof=1` again. The `norm` version mirrors the course's mean ± 2·SEM; the `t` version is what real small-sample work uses.

10. The Cheat Sheet

Everything above, compressed. If you reread one page before a quiz, this is the page.

Concept	One-line job	Watch out for
random.random()	one uniform number on [0,1]	seed once, not before every call
random.seed(n)	pins the whole sequence	reseeding rewinds to the start
random.gauss(mu, sigma)	one number from a bell	sigma sets width; mu just slides it
my_avg	sum / len	the warm-up for everything below
variance	mean of squared deviations / (n-1)	the -1 stays inside the parentheses
standard deviation	sqrt(variance), back in real units	a typical distance from the mean
ddof (numpy)	denominator = n - ddof	ddof=1 for samples; numpy defaults to 0
simulation / CLT	average many samples -> a bell	the data is invented, even the first value
SEM	SD / sqrt(n)	smaller than SD; quadruple n to halve it
95% CI	mean +/- 2 * SEM	'95% of such intervals', not this one

The five rules that prevent 90% of stats bugs

1. Seed once, then loop. Reseeding before each draw rewinds the sequence.
2. Divide variance by n minus one, and keep the minus-one inside the parentheses.
3. In NumPy, set ddof=1 for sample variance and SD; the default is population (n).
4. SEM shrinks like 1/sqrt(n): quadruple the sample to halve the error.
5. A confidence interval describes the procedure, not a probability about the one interval you built.

That’s the whole toolkit: random and seeds, the average, the Gaussian, variance and SD, simulation, standard error, and the confidence interval. Every number in this guide was executed and verified, not recalled. Now go forth and deviate responsibly.